



Esperanza Matemática Varianza

Marco Emilio Ramírez Palacios 0900-24-2624

Universidad Mariano Gálvez

Ingeniería en Sistemas

David Aarón Velázquez Zanunzini

10-5-2026

INDICE

Contenido

INDICE.....	2
I. INTRODUCCION.....	3
II. CONTENIDO	¡Error! Marcador no definido.
2.2. Concepto.....	¡Error! Marcador no definido.
2.3. Historia.....	¡Error! Marcador no definido.
III. BIBLIOGRAFIA.....	¡Error! Marcador no definido.
IV. ANEXOS.....	¡Error! Marcador no definido.

I. INTRODUCCION

El programa que realizamos para este proyecto busca desarrollar una biblioteca virtual, o algo similar la idea es que utilicemos los diferentes métodos que hemos aprendido en clase para implementarlos de manera organizada y en un proyecto unificado, donde utilizaremos listas enlazadas, colas, pilas, arboles AVL, grafos y menús aunque no los vimos, para ser sincero no pude completar el proyecto, no pude completar los grafos y a mi parecer estoy seguro que pude haber mejorado muchas cosas, como evitar que el programa se cierre cuando hay un error y la imposibilidad de ingresar números donde irían solo letras y algo que siento que si podría realizar pero me da miedo quedarme sin tiempo es el ISBN de los libros se puede repetir, y eso claramente es un error que se evita con una comparación, pero como digo el tiempo se agotó por la gran cantidad de proyectos y uno en específico el cual conlleva circuitos electrónicos que se llevó la gran parte del mismo.

II. CONCEPTOS

2.2.Variables

Las variables son aquellas clases que utilizamos para guardar cierta información como características, por ejemplo la clase libros contiene el nombre y categoría del libro, pero también tienen la utilidad de realizar ciertas acciones como en el caso de los nodos.

2.3.Listas enlazadas

Son una estructura de datos enlistados de manera lineal por decirlo de una manera, los datos los vamos apilando en una línea, y para buscar algún dato de la lista lo que realizamos es buscar dato por dato hasta encontrar el que buscamos.

2.4.Cola

Como bien lo indica su nombre este método nos permite realizar una cola, apilando los datos pero siempre con el orden de primero en entrar primero en salir, por lo que no nos sirve para usarlo como almacenamiento como en el caso de la lista enlazada pero si como una cola de espera.

2.5.Árbol AVL

El árbol binario una estructura de datos que permite reducir el tiempo de búsqueda, esto gracias a la manera de funcionar, donde crea una estructura con una raíz y dos ramas como máximo, de esta manera el árbol busca datos comparando de menor a mayor o depende como este configurado pero el nuestro está configurado de esa manera, la cosa es que con esa comparación puede reducir el tiempo de búsqueda y aunque aumentemos la cantidad de libros este podrá seguir buscándolos rápidamente.

2.6.Pila

La pila es el lado contrario de la cola, en vez de sacar el último dato que tiene saca al último que ha ingresado, en nuestro caso lo utilizamos para mostrar desde la ultima acción hasta la mas antigua.

2.7.Main o Menú

Esta es nuestro último método el cual contiene toda la información que presentaremos al usuario, a decir verdad sé que es importante para el programa para poder funcionar, pero podemos crearlo o mostrarlo de otras maneras para el usuario creo yo, pero en nuestro caso contiene cada una de las interacciones que tendremos con el usuario, donde de paso llamaremos a las funciones que realizamos con anterioridad, utilizando un menú para que el usuario solo ingrese un valor numérico para poder ingresar en cada caso.

2.8.Grafo

El grafo es un método que permite crear una estructura de datos conectados, estas conexiones se les conocen como caminos y determinan si un dato del grafo puede llegar hacia otro dato o no.

Hash map

Casi se me olvidaba, el hash map tiene la capacidad de almacenar varios datos como una lista, con la peculiaridad de que solo necesita una referencia para encontrar a un usuario, así que la búsqueda es prácticamente instantánea

III. PROGRAMA

Para esta parte quería pegar las partes del programa para que se pudiera ver mejor a lo que me refiero, pero queda muy mal se trae el color del fondo de inteliij.

3.2.Libro

En esta variable contenemos los datos del libro, la identidad que nosotros le designamos, ya sea ISBN, que utilizamos como un identificador único, algo así como una identidad única, luego el título, la disponibilidad para los préstamos, la categoría o género y el usuario que esté esperando el libro, en tal caso que lo haya solicitado y no estuviera disponible, después tenemos la recursividad para evitar problemas de redundancia, por ultimo encontramos una parte del código que permite utilizar las características en otros programas, además de otras características como comprobar la disponibilidad del libro y definir si hay un usuario en lista de espera.

3.3.Usuario

El usuario contiene menos características ya que solo utilizamos las necesarias, el ID del usuario, el nombre del usuario, la recursividad y la opción para utilizar las características en otros programas.

3.4.Préstamo

La variable de préstamo la utilizamos no para realizar los métodos de prestar y devolver en el momento, sino para guardar los datos después de realizar los métodos, algo importante que debería haber explicado antes, la clase usuario contiene las características de, Id préstamo como identificador único, usuario que tiene el préstamo, el libro el cual tienen en posesión, fecha del préstamo y una fecha de devolución, tiene las dos características mencionadas anteriormente con la peculiaridad de que más abajo se encuentra una función diferente, la cual sirve para que el programa pueda leer correctamente la lista de los préstamos.

3.5.Nodo

En nuestro programa hay dos nodos, uno para AVL y otro para uso general.

Primero hablaremos del Nodo general, este consiste de dos características, el primero es dato, el cual aunque parezca que no tiene un tipo de dato asignado es porque utilizamos <T> el cual se adapta al tipo de dato que esté trabajando facilitándonos el uso para varios programas, después ya solo realizamos la recursividad, ahora bien no tiene la otra característica de los demás para poder ser usado en otros programas porque ya está asignados como públicos, y por qué no los demás? Porque es una práctica que se realiza para evitar que un usuario no pueda acceder directamente a los datos, esta práctica la realizábamos antes en C# creando diferentes clases y haciendo referencia solo en los programas en los que era necesario, como un método de seguridad, pero como el nodo está hecho para trabajar con datos rápidos como la RAM entonces no necesita ese tipo de seguridad.

Ahora seguimos con el nodo AVL, el cual tiene la misma idea de no estar en privado, y se diferencia por las características: libro, altura del árbol, lado de búsqueda, izquierda y derecha, por último en la recursividad asignamos la altura inicial como 1.

3.6.Lista préstamo

Se compone de varios componentes que pondré enlistados

3.6.1. Agregar Final

Agregamos los datos al final de la lista, y en (size==0) definimos si es o no el primer dato de la lista.

3.6.2. Imprimir

Imprimimos los datos para que el usuario lo pueda ver, definimos un constructor con el nombre de sb, este nos permitirá mostrar en pantalla los datos de manera ordenada, además que definimos si actual, el dato, no se encuentra vacío para mostrar algo, sino para que intentar mostrar algo.

3.7.Cola de espera

La cola de espera se compone de varias partes.

3.7.1. Encolar

Es donde ingresamos los datos hacia la cola de espera, donde primero comprobamos si el dato esta vacío para luego definir en primera instancia si es el primero con (size==0) lo que nos indicara si es el primero, para luego si no lo es cola.siguiete lo pasa para atrás de la cola, por ultimo aumenta el tamaño (size++).

3.7.2. Dequeue

Después de ingresar a alguien a la cola cuando a este le llegue la señal empezará a sacar a la siguiente persona en la cola, siempre que el libro sea devuelto

3.7.3. Peek

Una lectura de la persona que se encuentre en primera fila para recibir el libro, pero que no pude introducir por temas de tiempo

3.7.4. Imprimir cola

El mismo caso que el anterior, pero mostrando todas las personas que están en la cola, tampoco lo introduje por tiempo

3.8.Árbol AVL

3.8.1. Insertar raíz

Primero que todo necesitamos definir una raíz, para posteriormente realizar las comparaciones con la misma.

3.8.2. Insertar recursiva

Después de realizar la comparación y el balanceo se retorna el nodo

3.8.3. Comparación

Si es menor a cero se va hacia la izquierda y al contrario hacia la derecha, lo que al final nos retorna el nodo ya cambiado.

3.8.4. Balance

Aunque no reciban un nombre como método lo que si tienen una funcionalidad y es balancear el árbol para que este sea lo más eficiente posible, por cierto el balanceo y comparación están dentro de la inserción recursiva,.

3.8.5. Obtener altura

Si no es nulo retorna la altura del nodo.

3.8.6. Obtener balance

Lo mismo pero buscando que ambos tengan la misma altura, donde para que estén balanceados debe dar

3.8.7. Rotar derecha o izquierda

Este fue uno de los casos que no entendí en el examen, y es que para poder balancear el árbol es necesario mover las alturas de los nodos, ya sea desbalanceado por el lado derecho o por el lado izquierdo, para poder reposicionar la posición del nodo y que quede balanceado.

3.8.8. In orden

Imprime los nodos del árbol en orden

3.8.9. In orden recursivo

Imprime todos los datos en orden pero de manera más ordenada teniendo en cuenta el árbol ya ordenado

3.8.10. Buscar libro

Llama a la función buscar recursiva hasta encontrar el dato que se busca, y luego devolverlo, si es que lo encuentra, sino avisa que o está equivocado o no existe.

3.8.11. Buscar Recursiva

Aquí está la función donde buscamos el libro por medio del ISBN, primero buscando al inicio para luego buscarlo de izquierda a derecha.

3.9. Historial pila

3.9.1. Push

Compara si el valor ingresado no es nulo para luego ingresar el nuevo nodo a la cima, y así con los demás, ya que este los ingresa a todos en la cima como mencionábamos con anterioridad.

3.9.2. Pop

No lo llegue a aplicar pero es para deshacer una acción.

3.9.3. Peek

Busca el dato más reciente que ingresamos a la pila.

3.10. Hash map usuario

3.10.1. Registrar usuario

Lo primero que realiza es comparar el id del usuario para saber si está duplicado “aunque creo que no lo aplique bien”, para después de realizar la comprobación introducir al usuario en el hash map y conformar el registro.

3.10.2. Buscar usuario

Solo pregunta si el id usuario es igual a algún usuario y lo regresa, y si no solo retorna nulo

3.10.3. Mostrar usuario

Primero comprueba que hayan usuarios y después por medio de un for imprimiendo uno por uno y por ultimo nos da la cantidad total de usuarios.

3.11. Menú

El final del programa, no colocamos grafos por lo mencionado anteriormente, para este caso solo voy a mencionar unas partes nuevas, no todas las llamadas del programa, sino partes nuevas y unos datos que ya no quise mover por miedo a romper el programa.

3.11.1. Switch

Este tiene una función interesante, y es que podemos definir un menú, no tiene que ser oblicatoriamente numérico pueden ser palabras o símbolos, pero la idea es que uno escoja una opción y luego de escoger la opción y terminarla regresar al menú

3.11.2. Opción 1

Nos permite registrar, buscar e imprimir la cantidad de libros.

3.11.3. Opción 2

Nos permite registrar usuarios y mostrar a los usuarios

3.11.4. Opción 3

Registramos prestamos, devoluciones y vemos la lista de prestamos que hemos realizado

3.11.5. Opción 4

Llama a la función de pila para que muestre los resultados, que en este caso serian las acciones efectuadas

3.11.6. Opción 5

No logre terminarla

3.11.7. Opción 6

Salir del programa

IV. CONCLUSION

Fue complicado trabajar todos los proyectos a decir verdad, este en especial me a dejado cansado las noches resolviéndolo, aunque me gusto realizarlo y también el de electrónica me parece que no me organice bien, quisiera haber terminado el tema de grafos y haber agregado las correcciones que me hubieran gustado, pero por ahora me relajo un poco sabiendo que entregue el proyecto.

